Case Study 1: Progressive Electricity Billing

Your Python code calculates the electricity bill using different rates for different units consumed. Here is the assignment content in a simple format, suitable for your ABL report.

1. Problem Statement

Develop a Python program to calculate an electricity bill using a progressive billing system. Different rates are applied to different ranges of electricity consumption.

2. Objectives

- To calculate the electricity bill based on units consumed.
- To use if, elif, and else conditional statements.
- To apply different rates for different unit ranges.
- To display the final electricity bill.

3. Assumptions

Electricity units	Rate per unit
First 100 units	₹1.50
Next 100 units (101–200)	₹2.50
Next 100 units (201–300)	₹4.00
Above 300 units	₹6.00

The billing is progressive, meaning each rate applies only to the units within its range.

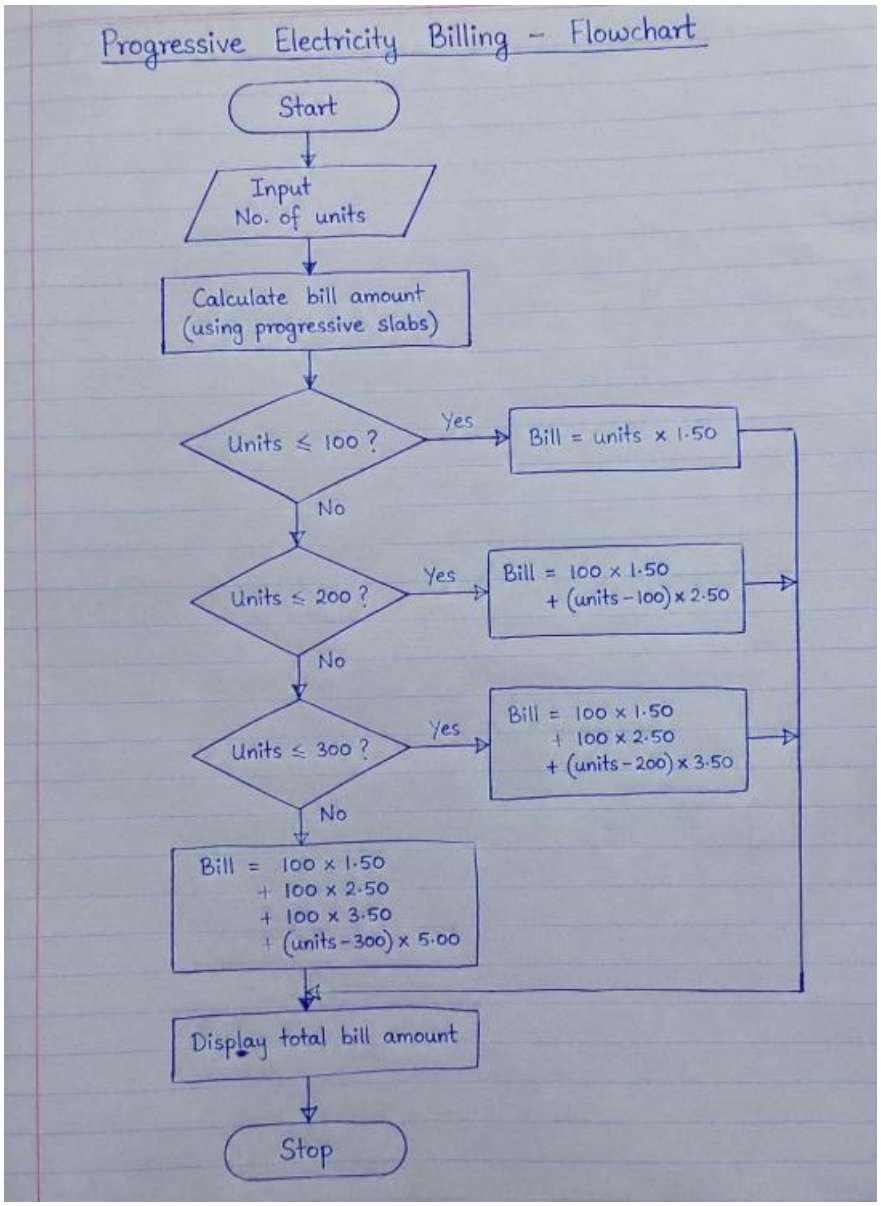
4. Algorithm

1. Start.
2. Read the number of electricity units consumed.
3. If units are less than or equal to 100, calculate the bill at ₹1.50 per unit.
4. Else, if units are less than or equal to 200, calculate the first 100 units at ₹1.50 and the remaining units at ₹2.50.
5. Else, if units are less than or equal to 300, calculate the first 100 units at ₹1.50, the next 100 at ₹2.50, and the remaining units at ₹4.00.
6. Otherwise, calculate the first 300 units using their respective rates and the remaining units at ₹6.00.

7. Display the electricity bill.

8. Stop.

5. Flowchart



6. Pseudocode

START

INPUT units

IF units <= 100

bill = units * 1.50

ELSE IF units <= 200

```
    bill = 100 * 1.50 + (units - 100) * 2.50
ELSE IF units <= 300
    bill = 100 * 1.50 + 100 * 2.50
          + (units - 200) * 4.00
ELSE
    bill = 100 * 1.50 + 100 * 2.50
          + 100 * 4.00 + (units - 300) * 6.00
END IF

PRINT bill
STOP
```

7. Python Source Code

```
units = int(input("Enter electricity units consumed: "))

if units <= 100:
    bill = units * 1.50

elif units <= 200:
    bill = (100 * 1.50) + (units - 100) * 2.50

elif units <= 300:
    bill = (100 * 1.50) + (100 * 2.50) + (units - 200) * 4.00

else:
    bill = (100 * 1.50) + (100 * 2.50) + (100 * 4.00) + (units - 300) * 6.00

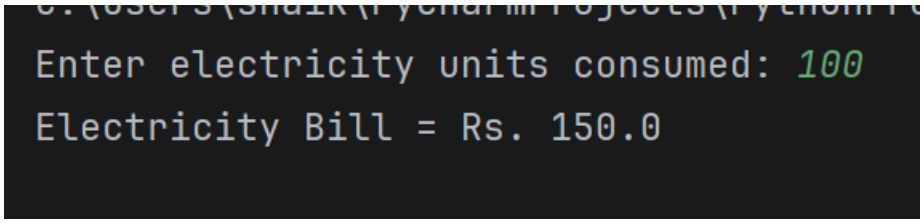
print("Electricity Bill = Rs.", bill)
```

8. Test Cases and Expected Output

Test case	Units consumed	Expected bill

1	50	₹75.00
2	150	₹275.00
3	250	₹600.00
4	350	₹1,100.00

9. Output Screenshot



10. Conclusion

This program calculates the electricity bill using a progressive billing system. It demonstrates the use of conditional statements, arithmetic operations, and decision-making in Python. It helps students understand how different rates are applied to different ranges of electricity consumption.

Case Study 2: Hospital Billing and Patient Classification

This case study is prepared in the same ABL format as Progressive Electricity Billing, using simple English suitable for a first-year B.Tech student.

1. Problem Statement

Develop a Python program to calculate a patient's hospital bill based on the number of days admitted, room type, and medical service charges. Classify the patient according to the total bill amount.

2. Objectives

- To calculate hospital charges based on the room type and admission days.
- To include medical service charges in the bill.
- To classify patients based on the total bill.
- To apply Python conditional statements such as if, elif, and else.

3. Assumptions

The following rates are assumed for this case study.

Room type	Charge per day
General ward	₹1,000

Semi-private room	₹2,000
Private room	₹3,500

Patient classification based on the total bill:

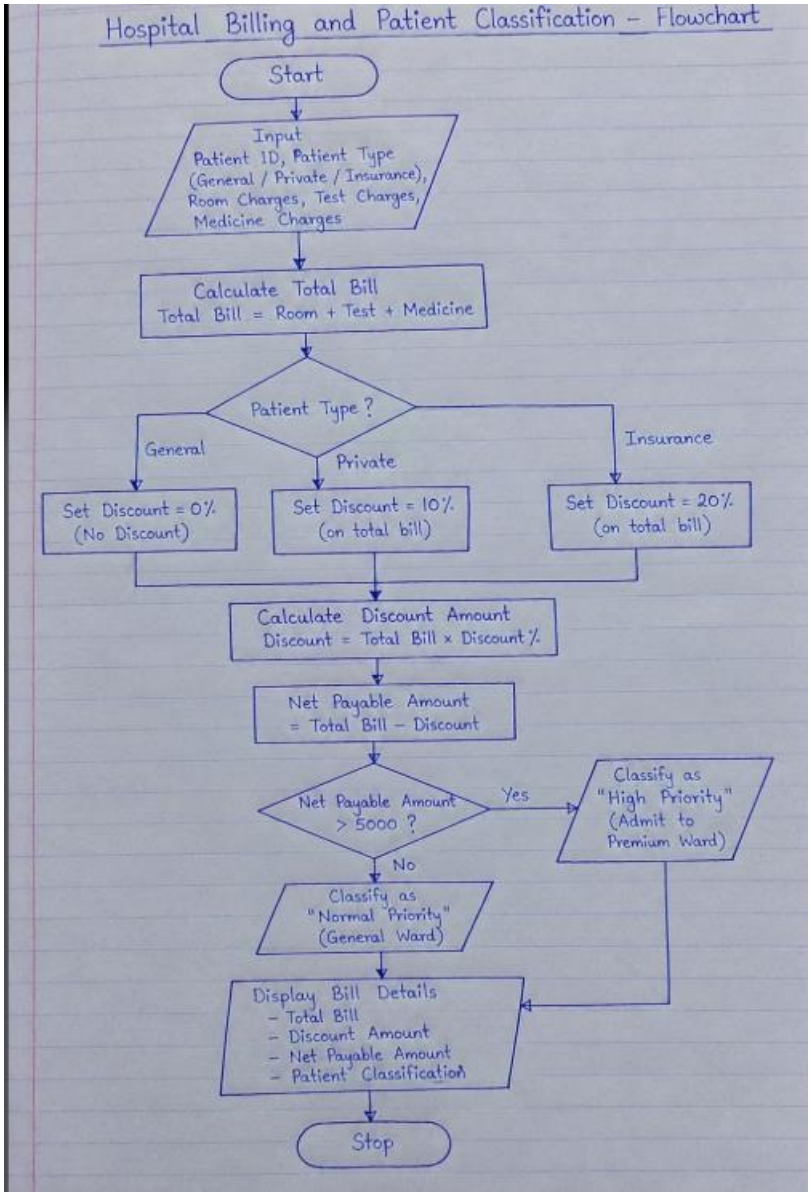
Total bill	Classification
Below ₹5,000	Basic
₹5,000–₹15,000	Moderate
Above ₹15,000	High-cost

Medical service charges are entered separately by the user. These rates are assumptions for the assignment, not actual hospital tariffs.

4. Algorithm

1. Start.
2. Enter the patient's name, number of admission days, room type, and medical service charges.
3. Calculate room charges based on the selected room type.
4. Calculate the total bill by adding room charges and medical service charges.
5. If the total bill is below ₹5,000, classify the patient as Basic.
6. Else, if the total bill is less than or equal to ₹15,000, classify the patient as Moderate.
7. Otherwise, classify the patient as High-cost.
8. Display the room charges, total bill, and patient classification.
9. Stop.

5. Flowchart



6. Pseudocode

START

INPUT patient_name

INPUT admission_days

INPUT room_type

INPUT medical_charges

IF room_type == 1

    room_charges = admission_days * 1000

```
ELSE IF room_type == 2
    room_charges = admission_days * 2000
ELSE
    room_charges = admission_days * 3500
END IF

total_bill = room_charges + medical_charges

IF total_bill < 5000
    category = "Basic"
ELSE IF total_bill <= 15000
    category = "Moderate"
ELSE
    category = "High-cost"
END IF

DISPLAY room_charges
DISPLAY total_bill
DISPLAY category

STOP
```

7. Python Source Code

```
print("HOSPITAL BILLING SYSTEM")
print("1. General Ward - Rs.1000 per day")
print("2. Semi-Private - Rs.2000 per day")
print("3. Private Room - Rs.3500 per day")

name = input("Enter patient name: ")
days = int(input("Enter number of admission days: "))
```

```
room_type = int(input("Enter room type (1/2/3): "))
medical_charges = float(input("Enter medical service charges: "))

if days <= 0 or medical_charges < 0:
    print("Invalid admission days or medical charges.")

elif room_type not in [1, 2, 3]:
    print("Invalid room type.")

else:
    if room_type == 1:
        room_charges = days * 1000
        room_name = "General Ward"

    elif room_type == 2:
        room_charges = days * 2000
        room_name = "Semi-Private Room"

    else:
        room_charges = days * 3500
        room_name = "Private Room"

    total_bill = room_charges + medical_charges

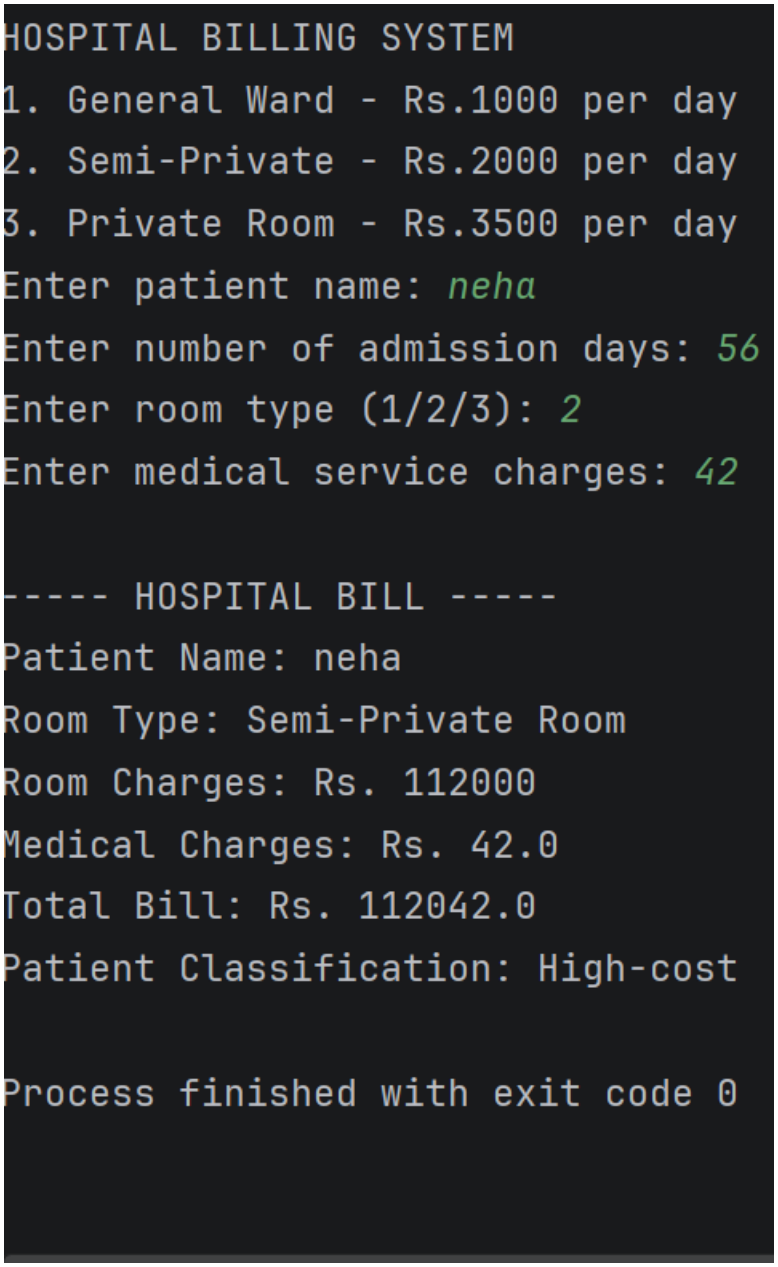
    if total_bill < 5000:
        category = "Basic"
    elif total_bill <= 15000:
        category = "Moderate"
    else:
        category = "High-cost"
```

```
print("\n----- HOSPITAL BILL -----")
print("Patient Name:", name)
print("Room Type:", room_name)
print("Room Charges: Rs.", room_charges)
print("Medical Charges: Rs.", medical_charges)
print("Total Bill: Rs.", total_bill)
print("Patient Classification:", category)
```

8. Test Cases and Expected Output

Test	Days	Room	Medical charges	Total bill	Classification
1	3	General	₹1,500	₹4,500	Basic
2	4	Semi-private	₹2,000	₹10,000	Moderate
3	5	Private	₹3,000	₹20,500	High-cost

9. Sample Output



10. Conclusion

This program calculates a patient's hospital bill based on admission days, room type, and medical service charges. It classifies patients into Basic, Moderate, and High-cost categories using conditional statements. It also demonstrates input validation and arithmetic operations in Python.

Case Study 3: Employee Payroll and Deduction Analysis

This case study follows the same ABL report format, with simple English, a handwritten-friendly flowchart, Python code, test cases, and sample output.

1. Problem Statement

Develop a Python program to calculate an employee's salary, deductions, and net salary based on the basic salary. The program should calculate allowances, apply deductions, and classify the employee according to the net salary.

2. Objectives

- To calculate gross salary using basic salary and allowances.
- To calculate deductions from gross salary.
- To determine the employee's net salary.
- To classify the salary using conditional statements.
- To understand arithmetic operations and decision-making in Python.

3. Assumptions

The following salary rules are used for this academic case study.

Component	Rate
House Rent Allowance (HRA)	20% of basic salary
Dearness Allowance (DA)	10% of basic salary
Provident Fund (PF) deduction	12% of basic salary
Tax deduction	5% of gross salary if gross salary exceeds ₹30,000

Formulas:

- Gross Salary = Basic Salary + HRA + DA
- Total Deductions = PF + Tax
- Net Salary = Gross Salary − Total Deductions

Net salary classification:

Net salary	Category
Below ₹20,000	Low
₹20,000–₹40,000	Medium
Above ₹40,000	High

These rates and categories are illustrative assumptions for the assignment, not official payroll or tax rules.

4. Algorithm

1. Start.

2. Enter the employee's name and basic salary.

3. Calculate HRA as 20% of basic salary.

4. Calculate DA as 10% of basic salary.

5. Calculate gross salary by adding basic salary, HRA, and DA.

6. Calculate PF as 12% of basic salary.

7. If gross salary exceeds ₹30,000, calculate tax as 5% of gross salary; otherwise, tax is zero.

8. Calculate total deductions by adding PF and tax.

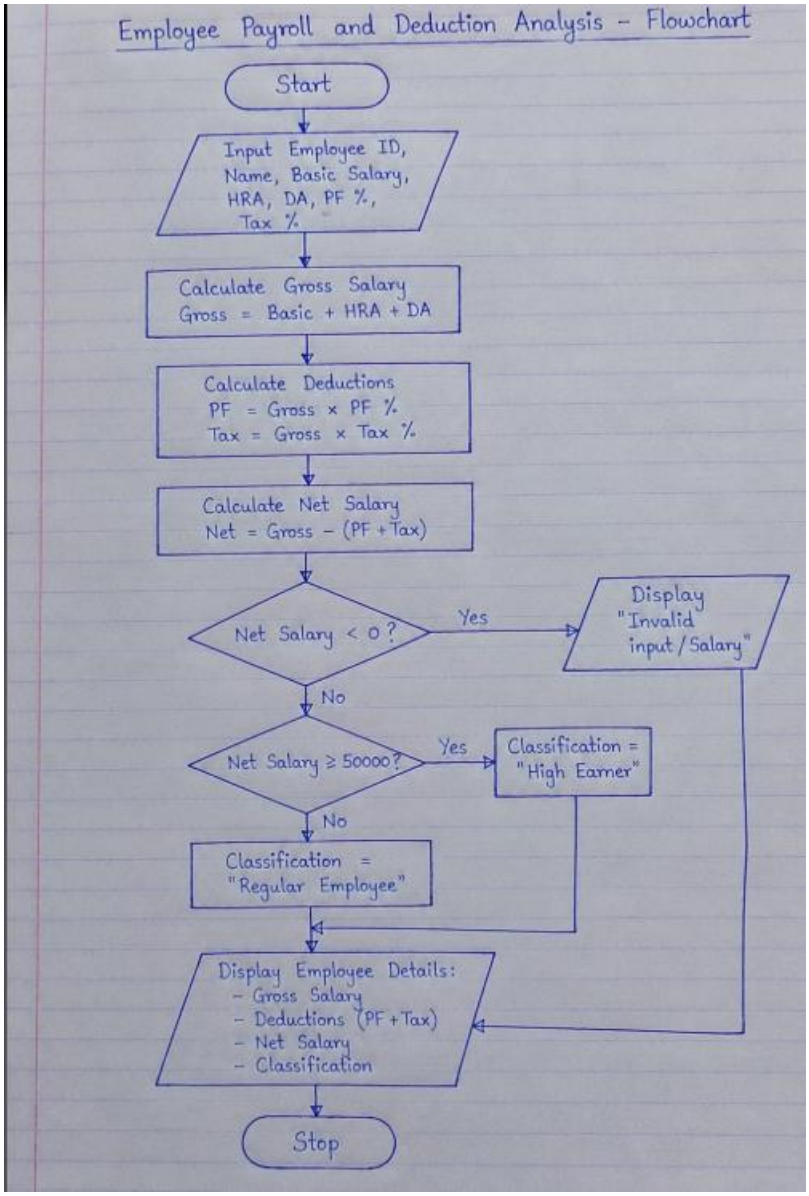
9. Calculate net salary by subtracting total deductions from gross salary.

10. Classify the net salary as Low, Medium, or High.

11. Display the salary details and classification.

12. Stop.

5. Flowchart



6. Pseudocode

START

INPUT employee_name

INPUT basic_salary

IF basic_salary <= 0

    PRINT "Invalid basic salary"

ELSE

    HRA = basic_salary * 0.20

```
    DA = basic_salary * 0.10

    gross_salary = basic_salary + HRA + DA
    PF = basic_salary * 0.12

    IF gross_salary > 30000
        tax = gross_salary * 0.05
    ELSE
        tax = 0
    END IF

    total_deductions = PF + tax
    net_salary = gross_salary - total_deductions

    IF net_salary < 20000
        category = "Low"
    ELSE IF net_salary <= 40000
        category = "Medium"
    ELSE
        category = "High"
    END IF

    DISPLAY salary details
END IF

STOP
```

7. Python Source Code

```
print("EMPLOYEE PAYROLL SYSTEM")
```

```
name = input("Enter employee name: ")
basic = float(input("Enter basic salary: Rs. "))

if basic <= 0:
    print("Invalid basic salary")

else:
    hra = basic * 0.20
    da = basic * 0.10

    gross = basic + hra + da
    pf = basic * 0.12

    if gross > 30000:
        tax = gross * 0.05
    else:
        tax = 0

    deductions = pf + tax
    net_salary = gross - deductions

    if net_salary < 20000:
        category = "Low"
    elif net_salary <= 40000:
        category = "Medium"
    else:
        category = "High"

    print("\n----- PAYROLL DETAILS -----")
    print("Employee Name:", name)
```

```
    print("Basic Salary: Rs.", round(basic, 2))
    print("HRA: Rs.", round(hra, 2))
    print("DA: Rs.", round(da, 2))
    print("Gross Salary: Rs.", round(gross, 2))
    print("PF Deduction: Rs.", round(pf, 2))
    print("Tax Deduction: Rs.", round(tax, 2))
    print("Total Deductions: Rs.", round(deductions, 2))
    print("Net Salary: Rs.", round(net_salary, 2))
    print("Salary Classification:", category)
```

8. Test Cases

Test	Basic salary	Gross salary	Net salary	Category
1	₹15,000	₹19,500	₹17,700	Low
2	₹25,000	₹32,500	₹28,750	Medium
3	₹40,000	₹52,000	₹43,200	High

9. Sample Output

```
EMPLOYEE PAYROLL SYSTEM
Enter employee name: Neha
Enter basic salary: Rs. 1000

----- PAYROLL DETAILS -----
Employee Name: Neha
Basic Salary: Rs. 1000.0
HRA: Rs. 200.0
DA: Rs. 100.0
Gross Salary: Rs. 1300.0
PF Deduction: Rs. 120.0
Tax Deduction: Rs. 0
Total Deductions: Rs. 120.0
Net Salary: Rs. 1180.0
Salary Classification: Low

Process finished with exit code 0
```

10. Conclusion

This program calculates an employee's gross salary, deductions, and net salary. It uses arithmetic operations and conditional statements to calculate allowances, apply deductions, and classify salaries. This case study demonstrates how Python can be used to automate basic payroll calculations.

Case Study 4: Retail Billing with Multiple Discount Rules

This case study follows the same ABL assignment format, with simple English, an algorithm, a handwritten-friendly flowchart, pseudocode, Python code, test cases, and sample output.

1. Problem Statement

Develop a Python program to generate a retail shopping bill by calculating the total purchase amount and applying multiple discount rules based on the purchase amount and customer membership. The program should calculate the final bill after discounts and display the savings.

2. Objectives

- To calculate the total cost of purchased items.
- To apply discounts based on purchase amount and membership.
- To calculate the final payable amount.
- To display the total discount and customer savings.
- To use conditional statements and arithmetic operations in Python.

3. Assumptions

The following discount rules are used for this academic case study.

Condition	Discount
Purchase amount ₹5,000 or more	10%
Purchase amount ₹2,000–₹4,999.99	5%
Purchase amount below ₹2,000	No purchase discount
Member customer	Additional 5% discount on the amount remaining after the purchase discount

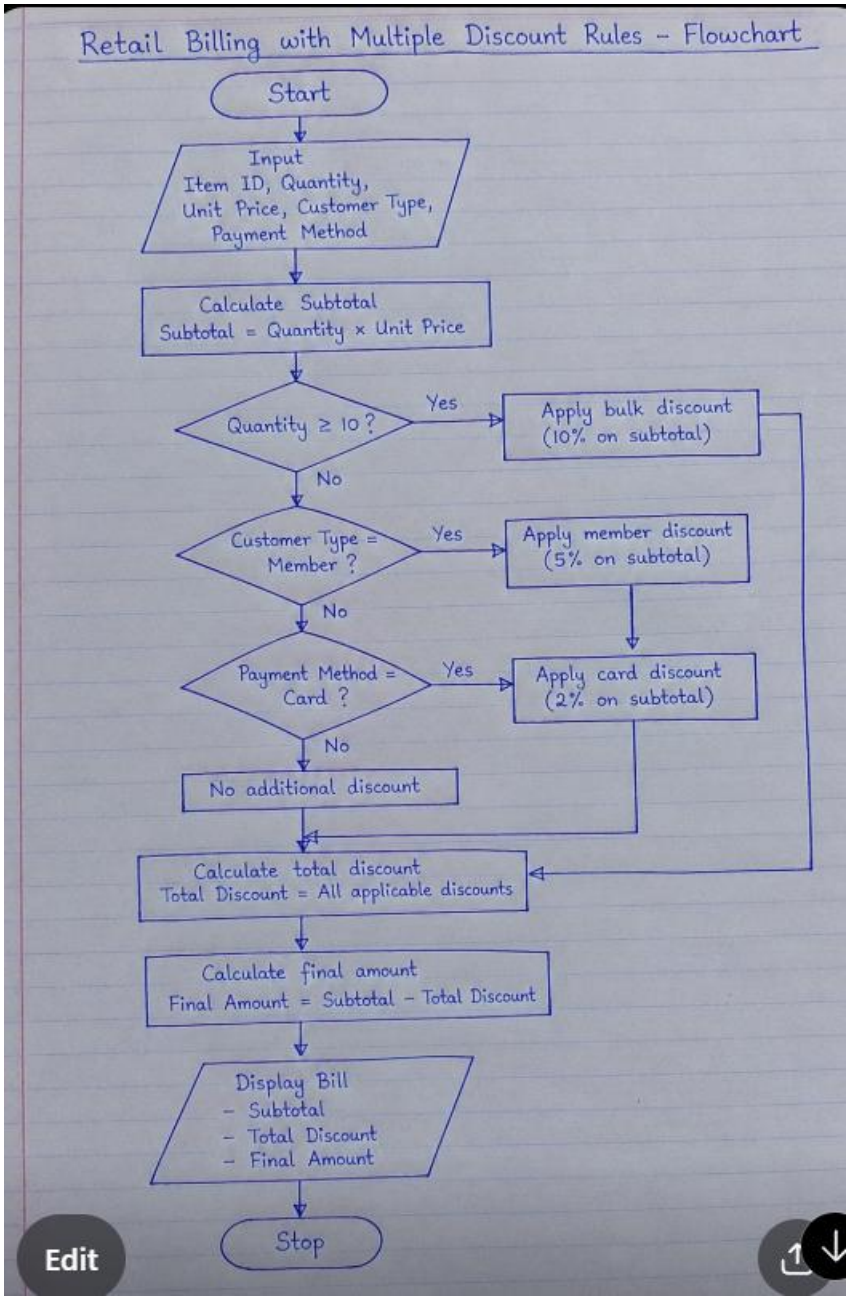
The membership discount is applied after the purchase discount. These are assumed rules for the assignment, not actual store policies.

4. Algorithm

1. Start.
2. Enter the customer's name and total purchase amount.
3. Ask whether the customer is a member.
4. If the purchase amount is ₹5,000 or more, calculate a 10% purchase discount.
5. Else, if the purchase amount is ₹2,000 or more, calculate a 5% purchase discount.
6. Otherwise, set the purchase discount to zero.
7. Subtract the purchase discount from the original amount.
8. If the customer is a member, calculate an additional 5% discount on the remaining amount; otherwise, set the membership discount to zero.
9. Calculate the final bill and total savings.
10. Display the bill details.

11. Stop.

5. Flowchart



7. Python Source Code

```
print("RETAIL BILLING SYSTEM")

name = input("Enter customer name: ")

amount = float(input("Enter total purchase amount: Rs. "))
```

```
if amount < 0:
    print("Invalid purchase amount")

else:
    membership = input(
        "Is the customer a member? (yes/no): "
    ).strip().lower()

    if membership not in ["yes", "no"]:
        print("Invalid membership status")

    else:
        # Calculate purchase discount
        if amount >= 5000:
            purchase_discount = amount * 0.10
        elif amount >= 2000:
            purchase_discount = amount * 0.05
        else:
            purchase_discount = 0

        # Amount after purchase discount
        remaining = amount - purchase_discount

        # Calculate membership discount
        if membership == "yes":
            member_discount = remaining * 0.05
        else:
            member_discount = 0
```

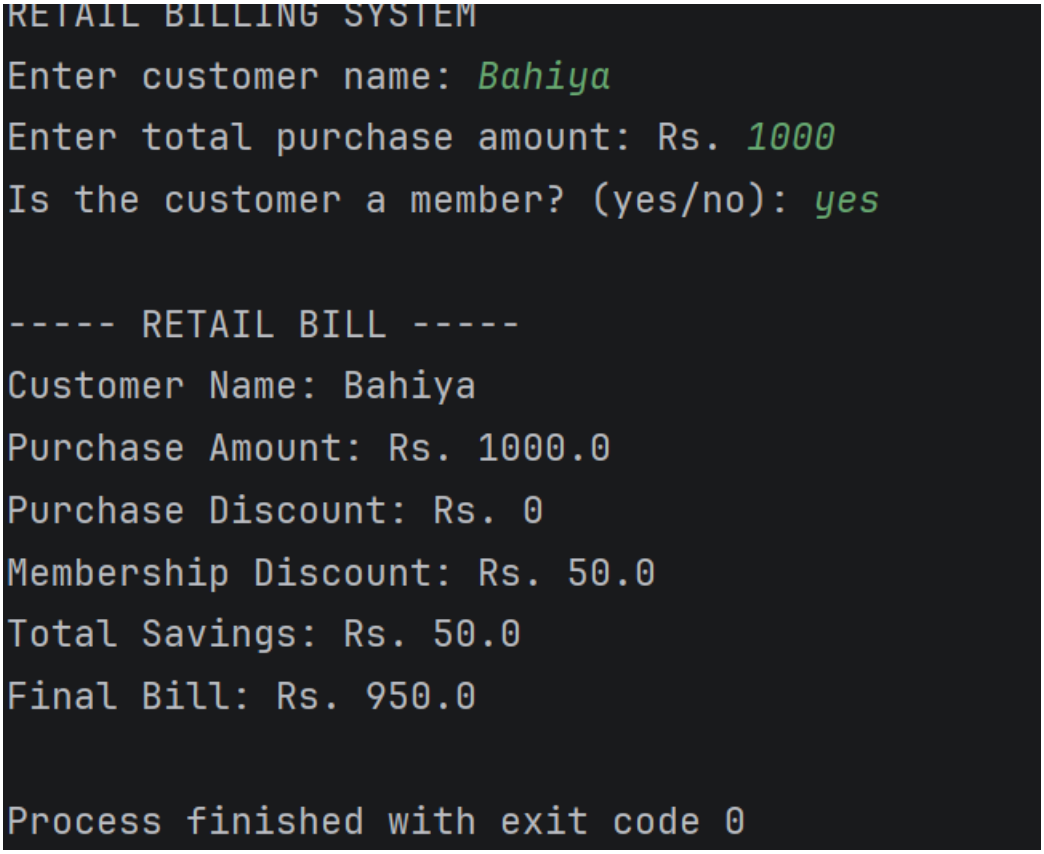
```
total_discount = purchase_discount + member_discount
final_bill = amount - total_discount

# Display the bill
print("\n----- RETAIL BILL -----")
print("Customer Name:", name)
print("Purchase Amount: Rs.", round(amount, 2))
print("Purchase Discount: Rs.",
    round(purchase_discount, 2))
print("Membership Discount: Rs.",
    round(member_discount, 2))
print("Total Savings: Rs.",
    round(total_discount, 2))
print("Final Bill: Rs.", round(final_bill, 2))
```

8. Test Cases

Test	Purchase amount	Member?	Total discount	Final bill
1	₹1,500	No	₹0.00	₹1,500.00
2	₹3,000	No	₹150.00	₹2,850.00
3	₹6,000	Yes	₹870.00	₹5,130.00
4	₹2,000	Yes	₹195.00	₹1,805.00

10. Sample Output



10. Conclusion

This program calculates a retail shopping bill using multiple discount rules. It applies purchase-based discounts and an additional membership discount to determine the final payable amount. It demonstrates conditional statements, arithmetic operations, input validation, and real-world billing logic in Python.

Case study 5: Manufacturing Cost and Profitability Analysis

Step 1: Problem Statement

Develop a Python program to calculate the total manufacturing cost, cost per unit, total sales revenue, and profit or loss of a manufacturing company.

The program should accept the number of units produced, raw material cost, labour cost, overhead cost, and selling price per unit. It should then calculate the total cost and determine whether the company has a profit or loss.

Step 2: Objectives

1. To calculate the total manufacturing cost.
2. To calculate the cost of manufacturing one unit.
3. To calculate the total sales revenue.

4. To determine the profit or loss.
5. To display the manufacturing report clearly.

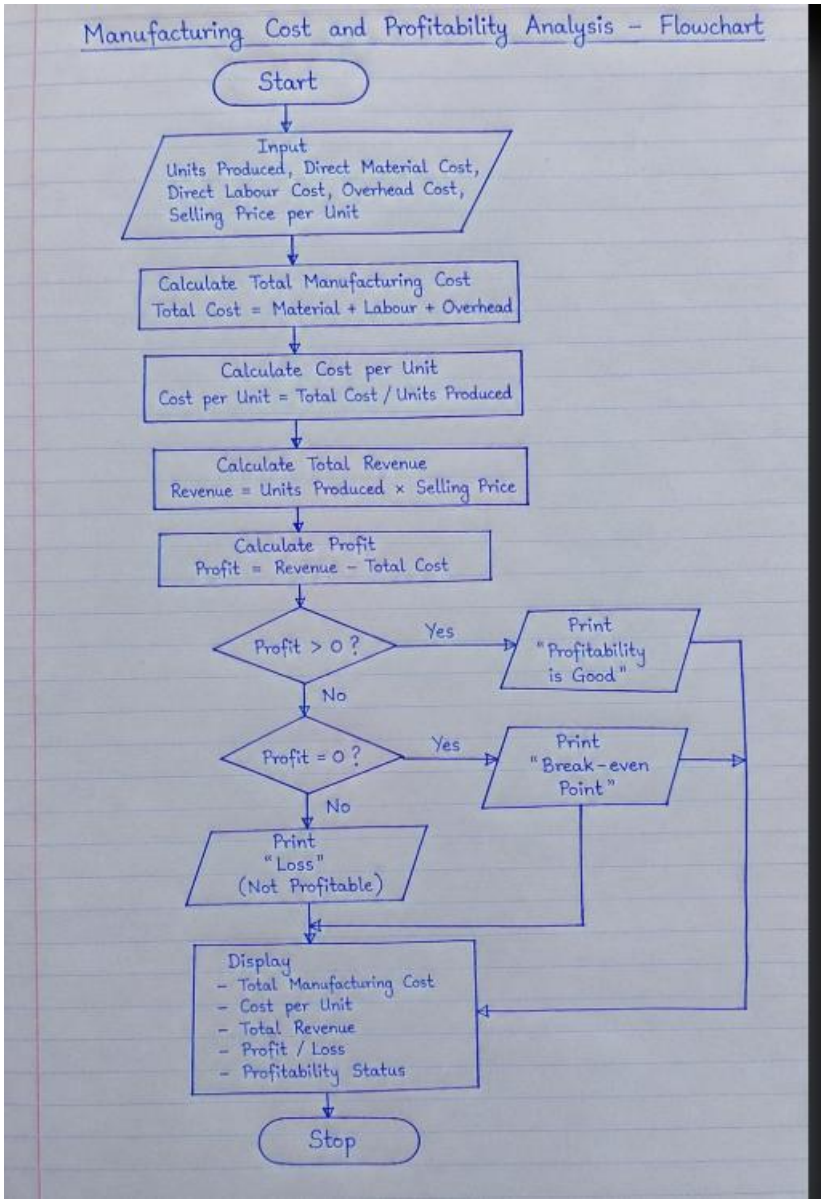
Step 3: Assumptions

- The number of units produced is greater than zero.
- All costs are entered as non-negative values.
- All manufactured units are sold.
- The selling price per unit remains constant.
- Raw material, labour, and overhead costs represent the total costs for the production batch.

Step 4: Algorithm

1. Start.
2. Input the number of units produced.
3. Input raw material cost, labour cost, and overhead cost.
4. Input the selling price per unit.
5. Calculate total manufacturing cost.
6. Calculate cost per unit.
7. Calculate total sales revenue.
8. Compare sales revenue with total manufacturing cost.
9. Display profit, loss, or break-even status.
10. Stop.

Step 5: flow chart



Step 6: Pseudocode

```
BEGIN
    INPUT units
    INPUT raw_material_cost
    INPUT labour_cost
    INPUT overhead_cost
    INPUT selling_price

    total_cost = raw_material_cost + labour_cost + overhead_cost
    cost_per_unit = total_cost / units
```

```
    revenue = units * selling_price

    IF revenue > total_cost
        profit = revenue - total_cost
        DISPLAY profit
    ELSE IF revenue < total_cost
        loss = total_cost - revenue
        DISPLAY loss
    ELSE
        DISPLAY "Break-even"
    END IF

    DISPLAY total_cost
    DISPLAY cost_per_unit
    DISPLAY revenue
END
```

Step 7: Python Program

```
units = int(input("Enter number of units produced: "))

if units <= 0:
    print("Units must be greater than zero.")
else:
    raw_material = float(input("Enter raw material cost: "))
    labour = float(input("Enter labour cost: "))
    overhead = float(input("Enter overhead cost: "))
    selling_price = float(input("Enter selling price per unit: "))

    if raw_material < 0 or labour < 0 or overhead < 0 or selling_price < 0:
        print("Costs and selling price cannot be negative.")
```

```
    else:
        total_cost = raw_material + labour + overhead
        cost_per_unit = total_cost / units
        revenue = units * selling_price

        print("\n--- Manufacturing Report ---")
        print("Total Manufacturing Cost:", total_cost)
        print("Cost Per Unit:", round(cost_per_unit, 2))
        print("Total Sales Revenue:", revenue)

        if revenue > total_cost:
            profit = revenue - total_cost
            print("Profit:", round(profit, 2))
        elif revenue < total_cost:
            loss = total_cost - revenue
            print("Loss:", round(loss, 2))
        else:
            print("Break-even: No profit, no loss.")
```

Step 8: Test Cases and Expected Output

Test	Units	Total cost	Selling price/unit	Expected result
1	100	₹8,000	₹100	Profit ₹2,000
2	100	₹12,000	₹100	Loss ₹2,000
3	100	₹10,000	₹100	Break-even

For Test 1, enter raw material cost ₹5,000, labour cost ₹2,000, overhead cost ₹1,000, and selling price ₹100.

Step 9: Sample output:

```
Enter number of units produced: 20
Enter raw material cost: 80
Enter labour cost: 90
Enter overhead cost: 100
Enter selling price per unit: 150

--- Manufacturing Report ---
Total Manufacturing Cost: 270.0
Cost Per Unit: 13.5
Total Sales Revenue: 3000.0
Profit: 2730.0

Process finished with exit code 0
```

Step 10: Conclusion

This Python program helps a manufacturing company understand its production costs, selling revenue, and profitability. It uses variables, arithmetic operators, conditional statements, input/output operations, and basic validation to support decision-making.

Case study 6: Multi-Currency Travel Budget Analysis

Step 1: Problem Statement

Develop a Python program to calculate a travel budget involving multiple currencies. The program accepts travel expenses in different currencies, converts them into Indian Rupees (INR) using the given exchange rates, calculates the total travel cost, compares it with the available budget, and displays the remaining balance or excess amount.

Step 2: Objectives

1. Accept travel expenses in different currencies.
2. Convert expenses into Indian Rupees.
3. Calculate the total travel expenditure.
4. Compare total expenses with the travel budget.
5. Display the remaining budget or overspending amount.

Step 3: Assumptions

- The budget is entered in Indian Rupees (INR).
- Exchange rates are entered by the user.
- All exchange rates must be greater than zero.
- All expenses and the budget must be non-negative.
- Exchange rates are assumed to remain constant during the calculation.

Step 4: Algorithm

1.Start.

2.Input the travel budget in INR.

3. Input expenses in USD, EUR, and GBP.

4.Input the exchange rate for each currency against INR.

5.Validate the input values.

6. Convert each expense into INR.

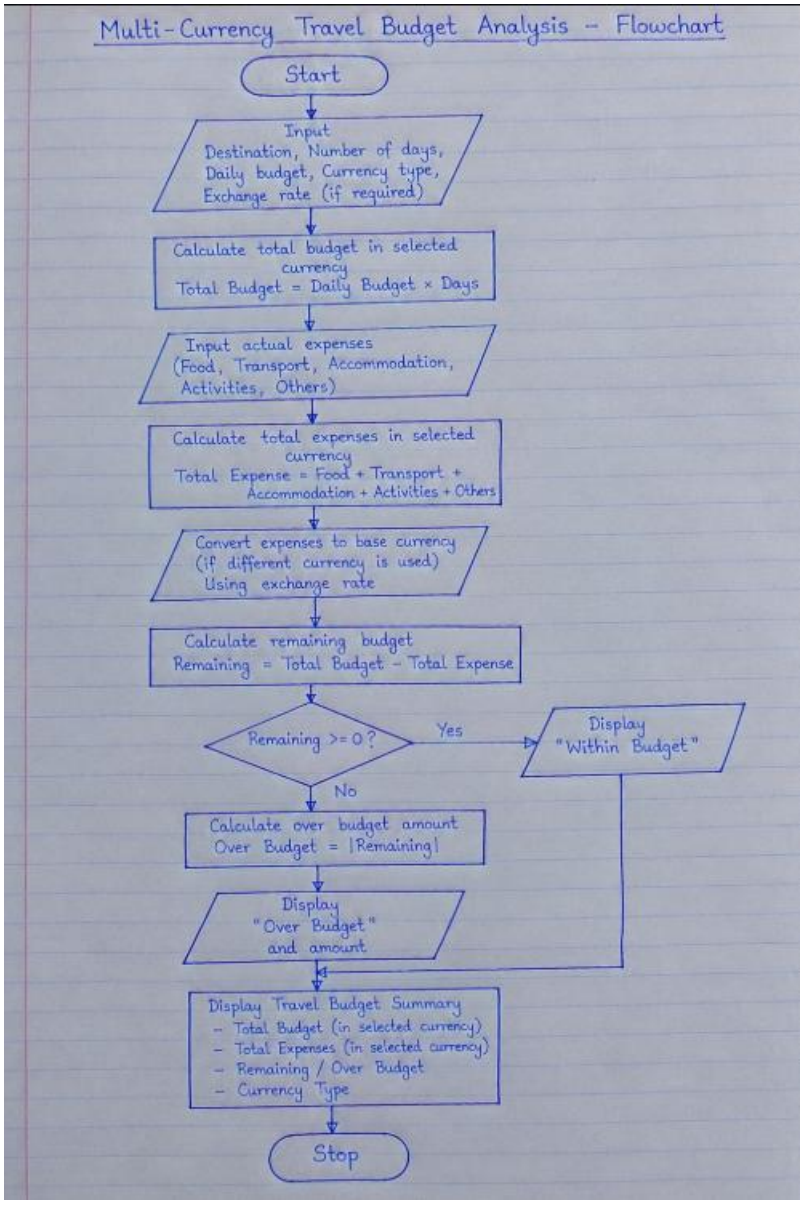
7. the total travel expenditure.

8.Compare the expenditure with the budget.

9.Display the converted expenses, total cost, and budget status.

10. Stop.

Step 5: Flowchart



Step 6: Pseudocode

BEGIN

INPUT budget

INPUT USD expense and exchange rate

INPUT EUR expense and exchange rate

INPUT GBP expense and exchange rate

IF inputs are invalid

```
        DISPLAY "Invalid input"
    ELSE
        USD_INR = USD expense * USD rate
        EUR_INR = EUR expense * EUR rate
        GBP_INR = GBP expense * GBP rate

        total = USD_INR + EUR_INR + GBP_INR

        DISPLAY converted expenses
        DISPLAY total

        IF total > budget
            DISPLAY overspending amount
        ELSE
            DISPLAY remaining budget
        END IF
    END IF
END
```

Step 7: Python Program

```
budget = float(input("Enter travel budget in INR: "))

usd = float(input("Enter expense in USD: "))
usd_rate = float(input("Enter USD to INR exchange rate: "))
```

```
eur = float(input("Enter expense in EUR: "))
eur_rate = float(input("Enter EUR to INR exchange rate: "))

gbp = float(input("Enter expense in GBP: "))
gbp_rate = float(input("Enter GBP to INR exchange rate: "))

if (budget < 0 or usd < 0 or eur < 0 or gbp < 0
        or usd_rate <= 0 or eur_rate <= 0 or gbp_rate <= 0):
    print("Invalid input. Check budget, expenses and exchange rates.")
else:
    usd_inr = usd * usd_rate
    eur_inr = eur * eur_rate
    gbp_inr = gbp * gbp_rate

    total = usd_inr + eur_inr + gbp_inr

    print("\n--- Travel Budget Report ---")
    print("USD expense in INR:", round(usd_inr, 2))
    print("EUR expense in INR:", round(eur_inr, 2))
    print("GBP expense in INR:", round(gbp_inr, 2))
    print("Total travel expenditure:", round(total, 2))

    if total > budget:
        print("Budget exceeded by INR:", round(total - budget, 2))
    elif total < budget:
```

```
        print("Remaining budget in INR:", round(budget - total, 2))
    else:
        print("Budget fully utilized.")
```

Step 8: Test Cases

For these examples, use illustrative exchange rates, not live market rates.

Test	Budget (INR)	Total expenses (INR)	Expected result
1	₹50,000	₹40,000	₹10,000 remaining
2	₹30,000	₹40,000	₹10,000 over budget
3	₹40,000	₹40,000	Budget fully utilized

Step 9: Sample Output

```
Enter travel budget in INR: 100
Enter expense in USD: 50
Enter USD to INR exchange rate: 5
Enter expense in EUR: 25
Enter EUR to INR exchange rate: 4
Enter expense in GBP: 30
Enter GBP to INR exchange rate: 4

--- Travel Budget Report ---
USD expense in INR: 250.0
EUR expense in INR: 100.0
GBP expense in INR: 120.0
Total travel expenditure: 470.0
Budget exceeded by INR: 370.0

Process finished with exit code 0
```

Step 10: Conclusion

The Multi-Currency Travel Budget Analysis program converts expenses from different currencies into Indian Rupees and calculates the total travel expenditure. It helps travellers manage their budgets and identify remaining funds or overspending. The program uses Python variables, arithmetic operators, conditional statements, input/output operations, and validation.

Case study7: Campus Event Financial Planning

Step 1: Problem Statement

Develop a Python program to prepare a financial plan for a college event. The program accepts the expected number of participants, registration fee per participant, and different event expenses such as venue, food, decorations, and other arrangements. It calculates the total income, total expenses, and profit or loss, and checks whether the event stays within its budget.

Step 2: Objectives

1. Calculate the total registration income.
2. Calculate the total event expenditure.
3. Determine the profit or loss.
4. Compare expenses with the allocated budget.
5. Display a complete financial report.

Step 3: Assumptions

1. The number of participants must be greater than zero.
2. Registration fees and expenses cannot be negative.
3. Every participant pays the same registration fee.
4. All expenses are entered in Indian Rupees (INR).
5. Sponsorship income is included if provided.

Step 4: Algorithm

1. Start.
2. Input the number of participants.
3. Input the registration fee per participant.
4. Input sponsorship income and the allocated budget.
5. Input venue, food, decoration, and other expenses.
6. Validate all input values.

7. Calculate total registration income.

8. Calculate total income, including sponsorship.

9. Calculate total expenses.

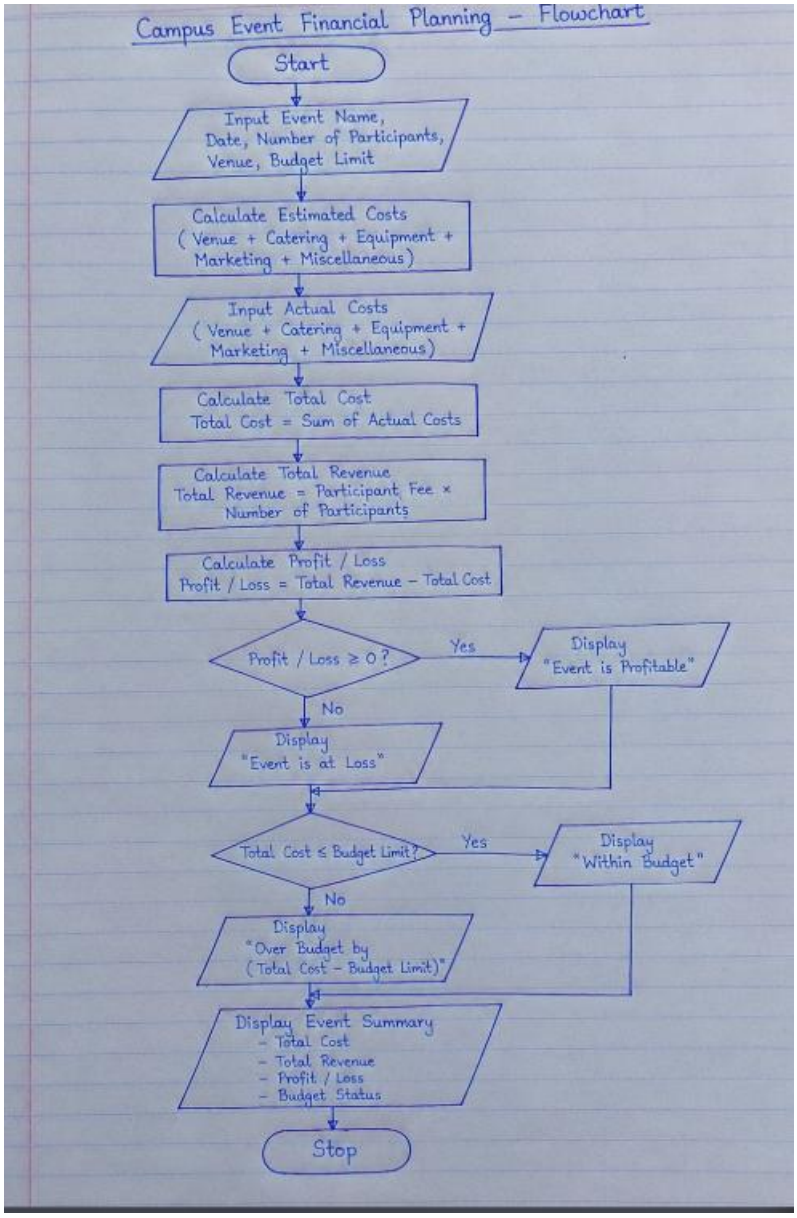
10. Calculate profit or loss.

11. Compare expenses with the allocated budget.

12. Display the financial report.

13. Stop.

Step 5: Flowchart



Step 6: Pseudocode

BEGIN

```
INPUT participants
INPUT registration_fee
INPUT sponsorship
INPUT budget
INPUT venue, food, decoration, other_expenses

IF any input is invalid
    DISPLAY "Invalid input"
ELSE
    registration_income = participants * registration_fee
    total_income = registration_income + sponsorship

    total_expenses = venue + food + decoration + other_expenses

    balance = total_income - total_expenses

    DISPLAY total_income
    DISPLAY total_expenses

    IF balance > 0
        DISPLAY "Profit"
    ELSE IF balance < 0
        DISPLAY "Loss"
    ELSE
        DISPLAY "Break-even"
    END IF

    IF total_expenses > budget
        DISPLAY "Budget exceeded"
    ELSE
```

```
            DISPLAY "Within budget"
        END IF
    END IF
END
```

Step 7: Python Program

```
print("CAMPUS EVENT FINANCIAL PLANNING")

participants = int(input("Enter number of participants: "))
registration_fee = float(input("Enter registration fee per participant: "))
sponsorship = float(input("Enter sponsorship amount: "))
budget = float(input("Enter allocated budget: "))

venue = float(input("Enter venue expenses: "))
food = float(input("Enter food expenses: "))
decoration = float(input("Enter decoration expenses: "))
other = float(input("Enter other expenses: "))

if (participants <= 0 or registration_fee < 0
        or sponsorship < 0 or budget < 0
        or venue < 0 or food < 0
        or decoration < 0 or other < 0):

    print("Invalid input. Check all entered values.")

else:
    registration_income = participants * registration_fee
    total_income = registration_income + sponsorship

    total_expenses = venue + food + decoration + other
```

```
    balance = total_income - total_expenses

    print("\n--- EVENT FINANCIAL REPORT ---")
    print("Registration Income: Rs.", round(registration_income, 2))
    print("Sponsorship Income: Rs.", round(sponsorship, 2))
    print("Total Income: Rs.", round(total_income, 2))
    print("Total Expenses: Rs.", round(total_expenses, 2))
    print("Allocated Budget: Rs.", round(budget, 2))

    if balance > 0:
        print("Profit: Rs.", round(balance, 2))
    elif balance < 0:
        print("Loss: Rs.", round(-balance, 2))
    else:
        print("Break-even: No profit, no loss.")

    if total_expenses > budget:
        print("Budget exceeded by Rs.",
              round(total_expenses - budget, 2))
    else:
        print("Event expenses are within budget.")
        print("Unused budget: Rs.",
              round(budget - total_expenses, 2))
```

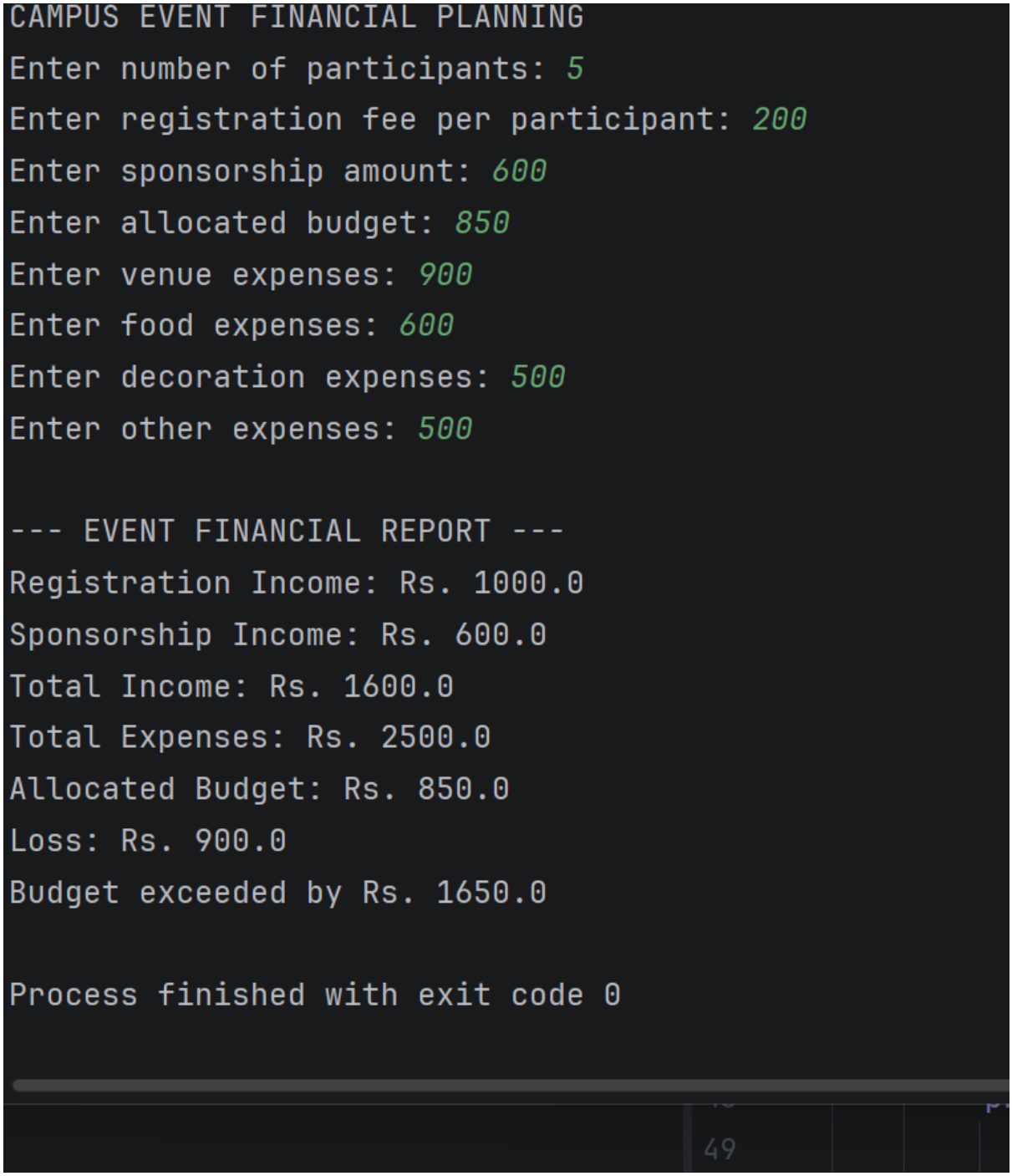
Step 8: Test Cases

Use these example values to test your program.

Test case	Total income	Total expenses	Expected result
1	₹50,000	₹35,000	Profit ₹15,000
2	₹30,000	₹40,000	Loss ₹10,000

3	₹40,000	₹40,000	Break-even

Step 9: Sample Output



Step 10: Conclusion

The Campus Event Financial Planning program helps college organizers manage event income, expenses, and budgets. It calculates profit or loss and identifies whether expenses exceed the allocated budget. The program uses Python variables,

arithmetic operators, conditional statements, input/output operations, and validation.

Case study 8: Examination Result and Grace-Mark Validation

Step 1: Problem Statement

Develop a Python program to analyze a student's examination results and validate eligibility for grace marks. The program accepts marks obtained in each subject, minimum passing marks, and the maximum grace marks allowed under a sample set of rules. It determines whether the student has passed, failed, or qualifies for grace marks.

Step 2: Objectives

1. Accept marks obtained in multiple subjects.
2. Check whether the student passes each subject.
3. Identify subjects in which the student has failed.
4. Check eligibility for grace marks.
5. Display the final result after applying the sample grace-mark rules.

Step 3: Assumptions

1. There are five subjects.

2. Each subject has a maximum of 100 marks.

3. The minimum passing mark is 40 in each subject.

4. A student can receive up to 5 grace marks in total.

5. Grace marks may be used to clear only one failed subject.

6. A student qualifies only if the shortfall is within the permitted grace marks.

7. These are sample rules; actual institutional policies may differ.

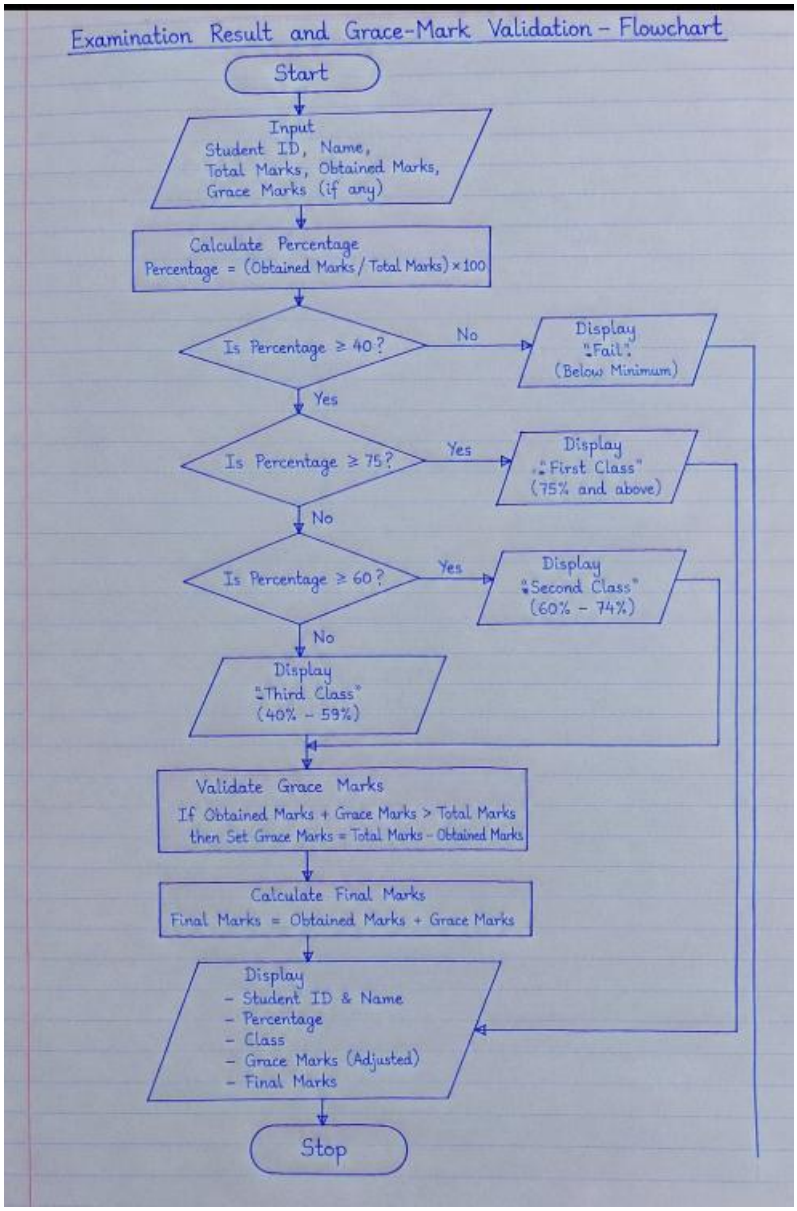
Step 4: Algorithm

1. Start.
2. Input the marks of five subjects.
3. Check whether all marks are between 0 and 100.
4. Identify the subjects with marks below 40.
5. If the student has no failed subjects, display “Pass”.
6. If exactly one subject is failed and the shortfall is at most 5 marks, apply the sample grace-mark rule.
7. Otherwise, display “Fail”.

8. Display the final marks and result.

9. Stop.

Step 5: Flowchart



Step 6: Pseudocode

```
BEGIN
    INPUT marks of five subjects


    IF marks are invalid
        DISPLAY "Invalid marks"
    ELSE
```

```
        FIND failed subjects

        IF no subjects failed
            DISPLAY "PASS"
        ELSE IF exactly one subject failed
            CALCULATE shortfall

            IF shortfall <= 5
                APPLY grace marks
                DISPLAY "PASS WITH GRACE MARKS"
            ELSE
                DISPLAY "FAIL"
            END IF
        ELSE
            DISPLAY "FAIL"
        END IF
    END IF
END
```

Step 7: Python Program

```
PASS_MARK = 40
MAX_GRACE = 5
SUBJECTS = 5

marks = []

for i in range(SUBJECTS):
    mark = int(input(f"Enter marks for subject {i + 1}: "))
    marks.append(mark)
```

```
if any(mark < 0 or mark > 100 for mark in marks):
    print("Invalid marks. Enter marks between 0 and 100.")

else:
    failed = []

    for i in range(SUBJECTS):
        if marks[i] < PASS_MARK:
            failed.append(i)

    grace_used = 0

    if len(failed) == 0:
        result = "PASS"

    elif len(failed) == 1:
        subject = failed[0]
        shortfall = PASS_MARK - marks[subject]

        if shortfall <= MAX_GRACE:
            grace_used = shortfall
            marks[subject] += grace_used
            result = "PASS WITH GRACE MARKS"
        else:
            result = "FAIL"

    else:
        result = "FAIL"

    print("\n--- EXAMINATION RESULT ---")
```

```
    for i in range(SUBJECTS):
        print(f"Subject {i + 1}: {marks[i]}")

    print("Grace marks used:", grace_used)
    print("Final Result:", result)
```

Step 8: Test Cases

Test	Marks in five subjects	Expected result
1	70, 65, 80, 55, 45	Pass
2	70, 65, 80, 55, 38	Pass with 2 grace marks
3	70, 65, 80, 55, 30	Fail
4	70, 65, 35, 55, 30	Fail

Step 9: Sample Output

```
Enter marks for subject 1: 90
Enter marks for subject 2: 80
Enter marks for subject 3: 85
Enter marks for subject 4: 95
Enter marks for subject 5: 80

--- EXAMINATION RESULT ---
Subject 1: 90
Subject 2: 80
Subject 3: 85
Subject 4: 95
Subject 5: 80
Grace marks used: 0
Final Result: PASS

Process finished with exit code 0
```

Step 10: Conclusion

The Examination Result and Grace-Mark Validation program checks student marks and determines the examination result using conditional statements and loops. It identifies failed subjects and applies the sample grace-mark rules when the student qualifies.

Case study 9: Attendance Recovery Computation

Step 1: Problem Statement

Develop a Python program to calculate a student's attendance percentage and determine how many additional classes the student must attend to reach the required attendance percentage.

The program accepts the total classes conducted, classes attended, and minimum required attendance percentage. It calculates the current attendance and the number of consecutive classes the student must attend to reach the required percentage.

Step 2: Objectives

1. Calculate the current attendance percentage.

2. Check whether the student meets the minimum attendance requirement.

3. Calculate the additional classes required to reach the target.

4. Display the student's attendance status.

5. Help students plan attendance recovery.

Step 3: Assumptions

1. Total classes conducted must be greater than zero.

2. Classes attended cannot exceed total classes conducted.

3. The target attendance percentage must be greater than 0 and at most 100.

4. The student attends every additional class during the recovery period.

5. No new absences occur during recovery.

Step 4: Algorithm

1. Start.

2. Input total classes conducted.

3. Input classes attended.

4. Input the target attendance percentage.

5. Validate the input values.

6. Calculate the current attendance percentage.

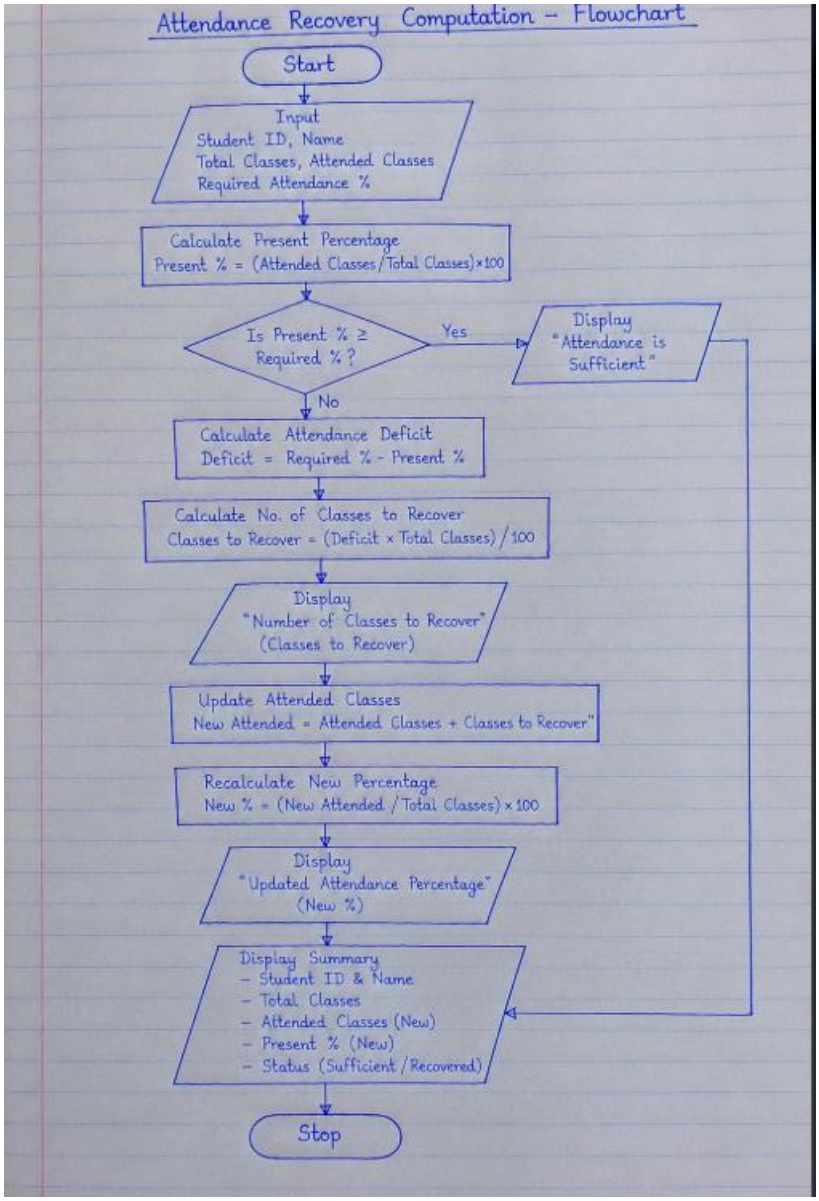
7. Check whether the student has already reached the target.

8. If not, calculate the additional classes required.

9. Display the current attendance and recovery details.

10. Stop.

Step 5: Flowchart



Step 6: Pseudocode

```
BEGIN
    INPUT total_classes
    INPUT attended_classes
    INPUT target_percentage

    IF total_classes <= 0 OR attended_classes < 0
       OR attended_classes > total_classes
       OR target_percentage <= 0
       OR target_percentage > 100
```

```
        DISPLAY "Invalid input"
    ELSE
        current = attended_classes / total_classes * 100

        DISPLAY current attendance

        IF current >= target_percentage
            DISPLAY "Target achieved"
        ELSE IF target_percentage == 100
            DISPLAY "Target cannot be reached by attendance alone"
        ELSE
            CALCULATE additional classes required
            DISPLAY additional classes required
        END IF
    END IF
END
```

Step 7: Python Program

```
import math

total_classes = int(input("Enter total classes conducted: "))
attended_classes = int(input("Enter classes attended: "))
target = float(input("Enter required attendance percentage: "))

if (total_classes <= 0 or attended_classes < 0
        or attended_classes > total_classes
        or target <= 0 or target > 100):

    print("Invalid input. Please check the entered values.")
```

```
else:
    current = attended_classes / total_classes * 100

    print("\n--- ATTENDANCE RECOVERY REPORT ---")
    print("Total classes:", total_classes)
    print("Classes attended:", attended_classes)
    print("Current attendance:", round(current, 2), "%")
    print("Required attendance:", target, "%")

    if current >= target:
        print("Status: Target attendance achieved.")

    elif target == 100:
        print("100% attendance cannot be reached by attending")
        print("future classes alone when classes were missed.")

    else:
        numerator = target * total_classes - 100 * attended_classes
        denominator = 100 - target

        additional = math.ceil(numerator / denominator)

        print("Additional classes required:", additional)
        print("Status: Attendance recovery needed.")
```

step 8: Test Cases

Test	Total classes	Attended	Target	Expected result
1	60	48	75%	Target achieved
2	60	42	75%	12 additional classes
3	100	60	75%	60 additional classes

4	60	30	100% Cannot reach 100% by future attendance alone

Step 9: Sample Output

```
Enter total classes conducted: 5
Enter classes attended: 4
Enter required attendance percentage: 50

--- ATTENDANCE RECOVERY REPORT ---
Total classes: 5
Classes attended: 4
Current attendance: 80.0 %
Required attendance: 50.0 %
Status: Target attendance achieved.

Process finished with exit code 0
```

Step 10: Conclusion

The Attendance Recovery Computation program calculates attendance percentage and determines how many additional classes are required to reach a target. It uses Python variables, arithmetic operations, conditional statements, input validation, and the math.ceil() function.

Case study 10: ATM Transaction Sequence Simulation

Step 1: Problem Statement

Develop a Python program to simulate an ATM transaction sequence. The program allows a user to check their account balance, deposit money, withdraw money, and exit the ATM system. It validates each transaction and ensures that withdrawals do not exceed the available balance.

Step 2: Objectives

1. Simulate basic ATM operations.
2. Display the current account balance.

3. Deposit money into the account.

4. Withdraw money after checking the balance.

5. Validate transactions and display suitable messages.

Step 3: Assumptions

- The account starts with a predefined balance.
- Deposit and withdrawal amounts must be positive.
- Withdrawals cannot exceed the available balance.
- The user can perform multiple transactions until choosing Exit.
- This is an educational simulation, not a real banking system.

Step 4: Algorithm

1. Start.

2. Initialize the account balance.

3. Display the ATM menu.

4. Ask the user to select an operation.

5. If the user selects Balance Enquiry, display the balance.

6. If the user selects Deposit, input the amount and update the balance.

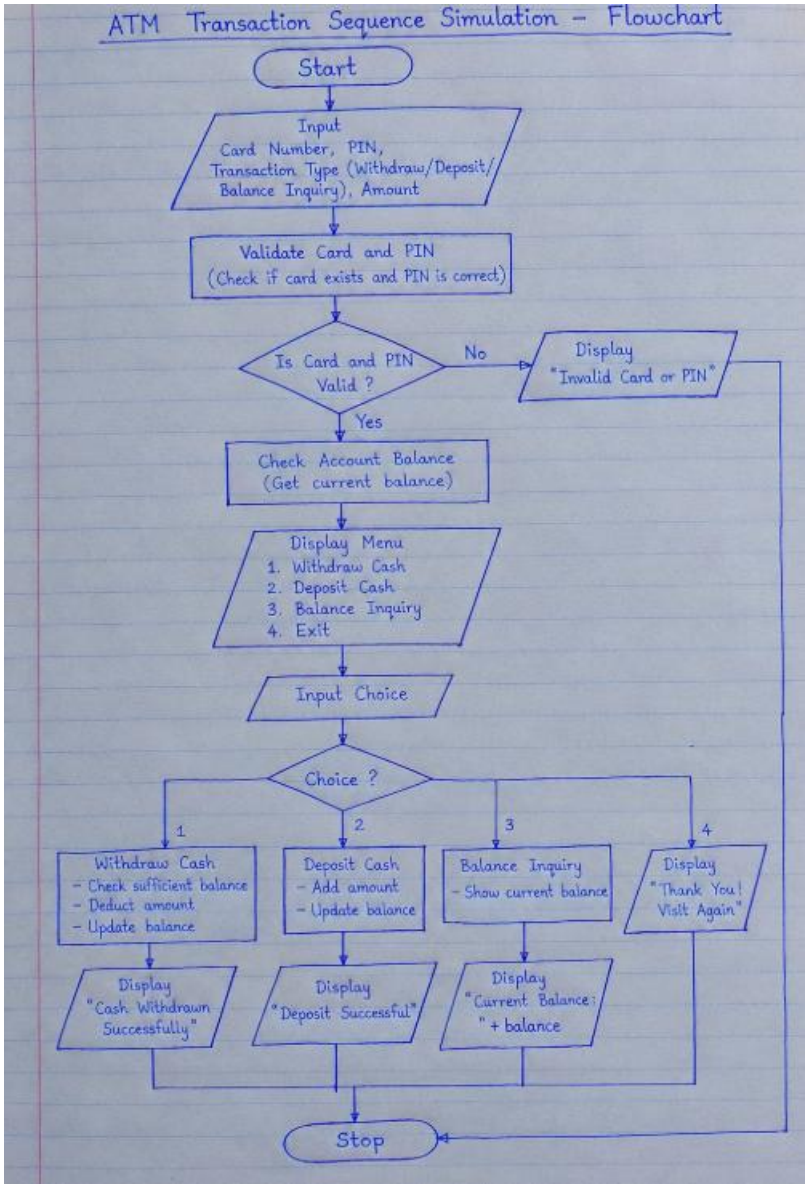
7. If the user selects Withdrawal, check the amount and available balance.

8. If the user selects Exit, terminate the program.

9. Otherwise, display an invalid choice message.

10. Repeat the menu until the user exits.

Step 5: Flowchart



Step 6: Pseudocode

```
BEGIN
    balance = 5000

    REPEAT
        DISPLAY ATM MENU
        INPUT choice

        IF choice == 1
            DISPLAY balance
```

```
        ELSE IF choice == 2
            INPUT deposit_amount
            IF deposit_amount > 0
                balance = balance + deposit_amount
                DISPLAY updated balance
            ELSE
                DISPLAY invalid amount
            END IF

        ELSE IF choice == 3
            INPUT withdrawal_amount
            IF withdrawal_amount <= 0
                DISPLAY invalid amount
            ELSE IF withdrawal_amount > balance
                DISPLAY insufficient balance
            ELSE
                balance = balance - withdrawal_amount
                DISPLAY updated balance
            END IF

        ELSE IF choice == 4
            DISPLAY goodbye message
            EXIT LOOP

        ELSE
            DISPLAY invalid choice
        END IF
    UNTIL choice == 4
END
```

Step 7: Python Program

```
balance = 5000.0

while True:
    print("\n--- ATM MENU ---")
    print("1. Balance Enquiry")
    print("2. Deposit Money")
    print("3. Withdraw Money")
    print("4. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        print("Available Balance: Rs.", round(balance, 2))

    elif choice == "2":
        amount = float(input("Enter deposit amount: Rs. "))

        if amount <= 0:
            print("Invalid amount. Enter a positive value.")
        else:
            balance += amount
            print("Deposit successful.")
            print("Updated Balance: Rs.", round(balance, 2))

    elif choice == "3":
        amount = float(input("Enter withdrawal amount: Rs. "))

        if amount <= 0:
```

```
            print("Invalid amount. Enter a positive value.")
        elif amount > balance:
            print("Insufficient balance.")
        else:
            balance -= amount
            print("Withdrawal successful.")
            print("Updated Balance: Rs.", round(balance, 2))

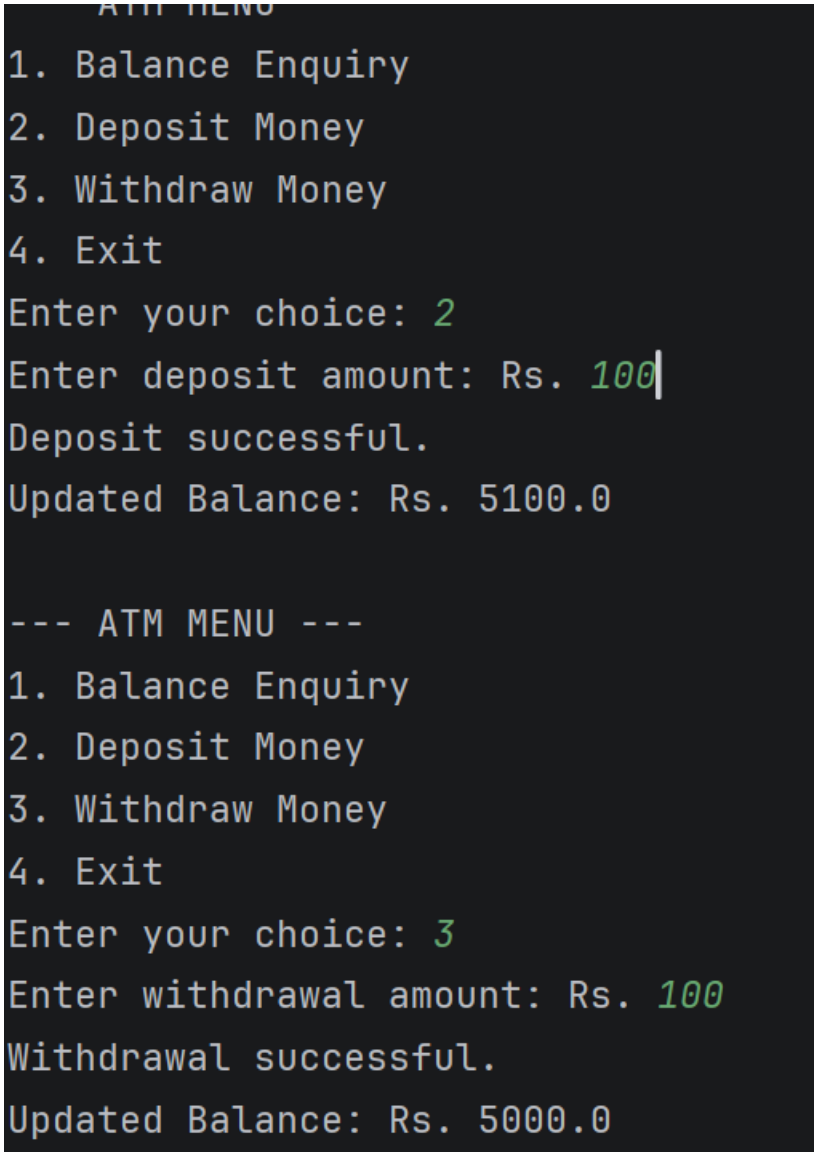
    elif choice == "4":
        print("Thank you for using the ATM.")
        break

    else:
        print("Invalid choice. Please try again.")
```

Step 8: Test Cases

Test	Operation	Input	Expected result
1	Balance enquiry	Choice 1	Balance ₹5,000
2	Deposit	₹2,000	Balance ₹7,000
3	Withdrawal	₹1,000	Balance ₹4,000
4	Withdrawal	₹6,000	Insufficient balance
5	Deposit	₹0	Invalid amount
6	Exit	Choice 4	Program terminates

Step 9: Sample Output



Step 10: Conclusion

The ATM Transaction Sequence Simulation demonstrates how Python can manage a sequence of banking operations. It uses variables, arithmetic operators, conditional statements, a while loop, and input validation to perform deposits, withdrawals, and balance enquiries.